Multi-Stage AI Workflow — Real IDE → Real CLI → Validate → GitHub

Author: Hope Hirwa Rukundo
Program: AmaliTech Apprenticeship — Fundamentals Track — Prompt Engineering Lab
Lab: Multi-Stage AI Workflow Across UX Types (Topic 7 Capstone)
Submission type: Resubmission (redo), following reviewer feedback on the original submission
Repository path: `submissions/03-multi-stage-ai-workflow/`
https://github.com/hrh2/Amalitech-fundamentals-labs/blob/main/Promp_Engineering/submissions/03-multi-stage-ai-workflow

Introduction

My name is Hope Hirwa Rukundo, and this document is the full technical write-up for my resubmission of the Multi-Stage AI Workflow lab, part of the Prompt Engineering module of the AmaliTech Apprenticeship program. The lab asks for a working automation that chains at least two different AI "UX types" — for example, a chat tool, an AI-enabled IDE, and an AI-driven CLI — so that the output of one genuinely becomes the input of the next, wired together with real orchestration and backed by clear documentation.

My original submission used n8n to call the same language model twice, once with a prompt telling it to behave like an IDE assistant and once with a prompt telling it to behave like a CLI generator. The reviewer correctly rejected this: it was one AI experience wearing two prompts, not two distinct AI UX types, and the pipeline committed whatever the

model produced straight to GitHub with no testing, no validation, and no evidence beyond a workflow export and a video.

This document explains, in full, what was wrong, how I redesigned the pipeline to use two genuinely different AI tools, how I re-engineered n8n's role to be an honest orchestrator rather than a disguised LLM caller, how I added real validation and error handling before anything reaches GitHub, and what I actually measured versus what I still need to measure myself. Everything below is written to be read end-to-end as a stand-alone report: the reasoning, the architecture, the code, the one real test run I performed while building this, and the exact steps still left for me to complete personally (opening WebStorm, running the live n8n workflow, and committing to GitHub with my own credentials).

Table of Contents

---

1. Objective

Chain two genuinely different AI UX types — an AI-enabled IDE and an AI CLI — into one workflow that turns a plain-language coding task into

committed, validated code, with n8n as the orchestration layer that connects the stages and enforces a quality gate before anything reaches GitHub.

2. Problem Statement

Turning a task description into a reviewed, working code change normally means: think through the approach, write the code, test it, then commit it — usually all inside one tool, by one person, serially. This workflow splits that into two AI-assisted stages that use *different UX paradigms* (an interactive IDE assistant for analysis, a scriptable CLI agent for generation) and automates the boring, error-prone middle part (running the generator, checking its output, committing only if it's actually valid) so a developer's manual effort is spent on the two places human judgement still matters: reviewing the IDE's plan, and reviewing the final diff/PR.

3. Why This Design — Redo Context

The previous submission used two `@n8n/n8n-nodes-langchain.chainLlm` nodes, both calling `gpt-5-mini`, differing only in a persona prompt ("IDE-based coding assistant" vs "CLI-focused code generator"). The reviewer correctly identified that as *not* two AI UX types — it was one UX type (an LLM API call inside n8n) wearing two prompts.

n8n Cloud (this workflow's host) cannot execute a local IDE or shell out to a locally-installed CLI binary — so the fix isn't to fake that capability inside n8n. Instead:

- **Stage 1 is a real, manual AI IDE session** (JetBrains WebStorm + AI Assistant). n8n's Stage 1 intake form *receives and structurally validates* the plan that session produces — it does not generate the plan.
- **Stage 2 is a real AI CLI**, invoked headlessly and locally via `scripts/run-stage2-cli.mjs`, which runs `claude -p` (Claude Code CLI, non-interactive/headless mode) against the

plan, then runs real syntax checks and (where a test script exists) real automated tests on what it generated, before POSTing the result to n8n.
- **n8n's only job is orchestration + a validation gate + committing/reporting** — exactly the role it can actually perform from the cloud, and exactly the role the lab needs it to perform.

This is a deliberate trade-off, not an evasion: full in-n8n automation of an IDE session is not possible (no headless API exists for JetBrains AI Assistant / Copilot / Cursor), and full in-n8n automation of the CLI stage is not possible on n8n Cloud specifically (no arbitrary local binary execution). Given those two hard constraints, this is the most automated, most honest architecture that still uses two genuinely distinct AI UX types. See `docs/architecture.md` for the full audit trail and rejected alternatives.

4. Architecture

```
[Developer, in WebStorm]                    [n8n Cloud]
 AI Assistant analyses task
 -> structured Plan JSON    ---(paste)-->  Stage 1 Intake
                                            -> Validate I
                                            -> shows Plan
                                               (nothing g

 Plan JSON + Task ID  -----(copy)-------------------------




                                           Stage 2 CLI R
                                           -> Validation
                                                YES -> S
                                                NO  -> F
                                                       -
```

Data moves as structured JSON at every hand-off: the IDE plan is JSON

(`summary`, `language`, `components`, `steps`), the CLI's result is JSON (`files[]`, `commitMessage`, `validation{}`). n8n never has to interpret free text. A rendered Mermaid version of this diagram, plus the full requirement-mapping and gap-analysis tables, is in `docs/architecture.md`.

5. Stage 1 — AI-Enabled IDE

- **Tool:** JetBrains WebStorm + AI Assistant (interactive chat panel in the editor).
- **How it's used:** the developer opens the target repo in WebStorm, pastes the task description into AI Assistant's chat, and asks it to analyse the task against the open project and produce a structured plan matching this shape:

```
{"summary": "...", "language": "...", "components":
```

- **Input:** task description, target runtime, and the open repository as live context (AI Assistant can see the project tree, not just pasted text — this is the actual value an IDE UX adds over a plain chat window).
- **Output:** the plan JSON above, pasted into the n8n Stage 1 Intake form.
- **Why this qualifies as an IDE AI UX:** it's an AI assistant embedded in, and aware of, a real code editor/project — inline chat, project-context-aware, distinct from both a standalone chat window and a terminal. This is the tool class the lab brief names as its own IDE example category (VS Code + Copilot is the brief's literal example; WebStorm + AI Assistant is the same UX class).
- **Evidence:** `evidence/stage-1-ide/` — **MANUAL ACTION REQUIRED**, see section 14.

6. Stage 2 — AI CLI

- **Tool:** Claude Code CLI, invoked headlessly (`claude -p ...`

`--permission-mode acceptEdits --output-format text`), via `scripts/run-stage2-cli.mjs`.

- **How it's invoked:** the script runs `claude -p "<prompt>"` with the working directory set to the output folder, so Claude's file-write tools operate directly on disk — this is a real, non-interactive CLI process, not an API call wrapped in a persona prompt.
- **Input:** the Stage 1 plan JSON, task description, and target runtime, assembled into a prompt by the script.
- **Output:** real files written to disk, then validated (syntax check per file type; `npm test` if a test script exists) and, on failure, regenerated up to `--max-attempts` times before giving up.
- **Why this qualifies as a CLI AI UX:** it is a genuine subprocess invocation of a published, installed AI CLI product operating non-interactively on a filesystem — the defining trait of a CLI UX (scriptable, headless, composable with other shell tools), as opposed to the IDE's interactive, project-aware chat panel.
- **Evidence:** `evidence/stage-2-cli/run-<taskId>.log` and `webhook-payload-<taskId>.json` — real, generated by an actual run (see section 11).

7. n8n's Role

n8n is the orchestration layer, not either AI UX. Concretely it:

1. Accepts and structurally validates the Stage 1 plan (rejects malformed JSON or missing required fields, causing a visible failed execution rather than silently passing bad data forward).
2. Accepts the Stage 2 result over a webhook.
3. Re-checks the validation flags the local script computed (`syntaxOk`, `testsPassed`, file count) before deciding what to do — this is the enforcement point, not a rubber stamp.
4. Commits each generated file to GitHub only if that check passes.
5. Otherwise files a GitHub issue describing exactly what failed, and does not commit.

8. Validation

Before anything is committed, two independent checks happen:

- **Locally (script), real:** per-file syntax check (`node --check` for JS, `bash -n` for shell, `python3 -m py_compile` for Python, `JSON.parse` for JSON), and `npm test` if the generated project has one. Results are captured verbatim in the log and included in the webhook payload.
- **In n8n (workflow), real:** the `Validation Passed?` IF node re-checks `syntaxOk`, `testsPassed`, and that at least one file was produced, independently of what the script claims — a payload that lies about passing but has an empty `files` array, for example, is still rejected.

9. Error Handling

- **Malformed Stage 1 plan:** `Validate IDE Plan` throws, the n8n execution fails visibly (shows in the n8n Executions list) instead of forwarding bad data.
- **Failed generation/validation:** `run-stage2-cli.mjs` retries generation up to `--max-attempts` (default 2) locally, feeding the previous failure's output back into the next prompt. See section 11 — this happened for real on the first test run.
- **Still failing after retries:** the payload is POSTed with `syntaxOk`/`testsPassed` set accurately. n8n's IF node routes to `Format Failure Report -> File Failure Issue`, which opens a GitHub issue with the task ID, attempt count, and full validation/test output, and responds `{"status":"rejected"}` — no commit happens.

10. GitHub Integration

`Commit To GitHub` (n8n GitHub node, OAuth2) creates/updates one file per generated file via the GitHub Contents API, using the commit message the CLI stage produced, and only runs on the validation-passed branch. On the failure branch, `File Failure Issue` uses the

same credential to open an issue instead.

11. Example Run

A real run of the Stage 2 mechanism (CLI generation, validation, retry, payload construction) was executed on 2026-09-03 using a hand-written test-fixture plan (explicitly **not** a WebStorm AI Assistant session — see section 13). Full details in `docs/run-log.md`. Summary:

- Task: "Add a Node.js CLI that validates a JSON file against a JSON schema and prints pass/fail"
- Attempt 1 failed (`npm test` hit `MODULE_NOT_FOUND` from an ambiguous test script path) — real failure, real detection, real retry.
- Attempt 2 passed: 8 files generated, all syntax-checked OK, 3/3 automated tests passed.
- Total wall time: 2m53.7s across both attempts including `npm install`/`npm test`.
- Output: `generated-output/sample-project/`. Log: `evidence/stage-2-cli/run-test-fixture-run-1.log`. Payload: `evidence/stage-2-cli/webhook-payload-test-fixture-run-1.json`.

The n8n Cloud half (posting this payload to the live webhook, watching the IF branch, and the resulting GitHub commit) has **not** been executed yet — that requires importing the workflow into my own n8n Cloud account. See section 14.

12. Efficiency

See `docs/efficiency-analysis.md` for full numbers. Headline: the Stage 2 CLI-generation + validation step, measured for real, took 2m53.7s end-to-end (including one failed attempt and a real retry). The manual-baseline comparison and the n8n-side (webhook → commit) timing are either estimated-and-labelled-as-such or pending my own measurement — not invented.

13. Limitations

- **Stage 1 is not automated inside n8n**, and cannot be, given the tools involved (no headless API for JetBrains AI Assistant / Copilot / Cursor). It's a real, evidenced manual step, not a gap being hidden — see `docs/architecture.md`.
- **Stage 2 does not run inside n8n Cloud** — it runs on the developer's machine because n8n Cloud cannot execute local binaries. n8n still owns the validation gate and the commit decision.
- **The example run's "Stage 1 plan" was a test fixture I wrote**, not a real WebStorm AI Assistant session, so it demonstrates the Stage 2 → n8n mechanism working, not a fully authentic Stage 1 → Stage 2 → GitHub run. Completing that requires a real WebStorm session and a real n8n Cloud execution, both still mine to do.
- **Retry is local-only** (the CLI script retries generation); n8n itself does not re-trigger Stage 2 — if validation fails twice, I have to fix the plan/task and re-run the script manually. An n8n-side retry would require the workflow to invoke Stage 2 itself, which the hosting constraint above rules out.
- **The "tests" run are whatever the CLI chose to generate** — for tasks where a cheap automated test isn't natural (e.g., pure infra scripts), only syntax checking runs, and `testOutput` says so explicitly rather than claiming a test suite that doesn't exist.

14. Manual Action Required — My Remaining Steps

1. **Import `workflow/multi-stage-ai-workflow.json`** into my n8n Cloud account and activate it. Grab the Stage 1 form URL and the Stage 2 webhook URL (Stage 2 CLI Result node) from n8n. This was not test-imported into a live n8n instance while drafting it, so node parameters/typeVersions should be treated as a best-effort draft — if a specific node (e.g. `Validation`

`Passed?`'s condition syntax, or the `Stage 1 Complete – Next Step` Form node) shows a type-version mismatch on import, open it in the n8n editor and let n8n's UI migrate/fix it; the logic described in `docs/architecture.md` is what matters, not the exact JSON shape.

2. **Run a real Stage 1 session**: open a target repo in WebStorm, use AI Assistant to analyse a real task, and get it to output the plan JSON shape shown above. Screenshot/record the chat.
3. **Submit that plan** through the Stage 1 Intake form (task description, target runtime, existing code if any, and the plan JSON). Save the Task ID it gives me.
4. **Run Stage 2 for real**, pointing at the live webhook:

```
node scripts/run-stage2-cli.mjs \
  --task-id <taskId from step 3> \
  --task-description "<same task description>" \
  --target-runtime "<same target runtime>" \
  --plan-file <path to the plan JSON, saved from the
  --out-dir generated-output/<new-sample-name> \
  --webhook-url <the Stage 2 webhook URL>
```

5. **Confirm in GitHub** that either the files were committed (success path) or an issue was filed (failure path), and note the commit SHA / issue URL.
6. **Fill in the real entries** in `docs/run-log.md` and `docs/efficiency-analysis.md` (n8n execution duration is visible in the n8n Executions panel for that run) — templates are already there.
7. **Capture evidence** per `evidence/*/README.md` in each subfolder, and re-export the workflow JSON from n8n after activation to replace `workflow/multi-stage-ai-workflow.json` with the live version (same content, but confirms it imported and ran cleanly).

15. Repository Structure

```
03-multi-stage-ai-workflow/
├── README.md                          <- this document
├── workflow/
│   ├── multi-stage-ai-workflow.json     <- revised, impo
│   └── previous-version-rejected.json   <- original subm
├── scripts/
│   └── run-stage2-cli.mjs             <- invokes Claude Code
├── docs/
│   ├── architecture.md                <- diagram, requiremen
│   ├── run-log.md                     <- real run + template
│   └── efficiency-analysis.md         <- measured vs. estima
├── evidence/
│   ├── stage-1-ide/README.md          <- what to capture fro
│   ├── stage-2-cli/                   <- real log + payload
│   ├── validation/README.md           <- what to capture fro
│   └── github/README.md               <- what to capture fro
├── generated-output/
│   └── sample-project/                <- real files generate
└── video.mp4                          <- original demonstrat
```